
UNIT 8 TREES

Structure

- 8.0 Introduction
- 8.1 Objectives
- 8.2 General Trees
- 8.3 Binary Trees
- 8.4 Implementing Trees
- 8.5 Tree Traversal Algorithms
- 8.6 Case Study: An Expression Tree
- 8.7 Summary
- 8.8 Further Readings

8.0 Introduction

Trees arrange elements in hierarchical structure. They provide an efficient way to represent and manage hierarchical relationships between data elements. Trees find extensive applications in file systems, database indexing, syntax parsing, and organizational hierarchies.

8.1 Objectives

After completing this unit, you will be able to:

- Understand the fundamental concepts and terminology of tree data structures
- Differentiate between general trees and binary trees
- Implement basic tree operations including insertion, deletion, and traversal
- Apply different tree traversal algorithms (pre-order, in-order, post-order, level-order)
- Create and evaluate expression trees
- Analyze the time complexity of various tree operations

8.2 General Trees

We use Trees to manage and identify hierarchical relationship between elements. Data that has parent-child relationships such as manager-employee, teacher-student, organization-sub organization can be stored in tree data structure.

In this section we closely look at the definition, structure and use cases for tree data structure.

GENERAL TREES: DEFINITION AND STRUCTURE

Nodes are main elements of tree data structure. Each node has zero or multiple child nodes. This flexibility allows general trees to represent various types of hierarchical data, such as file systems or organizational structures.

TERMINOLOGY

- Node: The fundamental part of a tree that contains data and links to its children.
- Root: The topmost node of the tree.
- Leaf: A node without any children.
- Subtree: A tree formed by a node and its descendants.

TRAVERSING GENERAL TREES

Traversal refers to visiting all nodes of the tree in a specific order. Common traversal methods include pre-order, post-order, and level-order traversals.

PRE-ORDER TRAVERSAL

Root node is visited first after which we visit the child nodes in pre-order traversal.

```
def pre_order_traversal(node):
    if node:
        print(node.data)
        for child in node.children:
            pre_order_traversal(child)
```

```
# Example usage
pre_order_traversal(root)
'''
```

POST-ORDER TRAVERSAL

All the child nodes are visited before the root node in post order traversal.

```
def post_order_traversal(node):
    if node:
        for child in node.children:
            post_order_traversal(child)
        print(node.data)
```

```
# Example usage
post_order_traversal(root)
'''
```

Sample Python code for Depth first travel and breadth first traversals are given below

```
def depth_first_traversal(node, level=0):
    """Print tree nodes in depth-first order"""
    print(" " * level + node.data)
    for child in node.children:
        depth_first_traversal(child, level + 1)
```

```
def breadth_first_traversal(root):
    """Print tree nodes in breadth-first order"""
    if not root:
        return
    queue = [root]
    while queue:
        current = queue.pop(0)
        print(current.data)
        queue.extend(current.children)
```

Node Operations

Common operations include adding child nodes, removing nodes, and searching for specific nodes:

```
class TreeNode:
```

```
    def __init__(self, data):
```

```
        self.data = data
```

```
        self.children = []
```

```
        self.parent = None
```

```
    def add_child(self, child):
```

```
        child.parent = self
```

```
        self.children.append(child)
```

```
    def remove_child(self, child):
```

```
        if child in self.children:
```

```
            child.parent = None
```

```
            self.children.remove(child)
```

PRACTICAL USE CASES OF GENERAL TREES

Given below are few real-world use cases for trees -

File System Organization

- Directories containing files and subdirectories
- Each directory can have multiple children

Organization Charts

- Representing company hierarchies
- Department structures

XML/HTML DOM

- Representing document structure with hierarchical structure

Menu Systems

- Application menu hierarchies
- Game skill trees

BEST PRACTICES

1. When implementing general trees, consider these best practices:
2. Memory Management
 - Implement proper cleanup methods
 - Handle circular references carefully
3. Performance Optimization
 - Choose appropriate traversal methods based on use case
 - Consider caching frequently accessed nodes
4. Data Validation
 - Validate node relationships before modifications
 - Implement error checking for operations
5. Design Considerations
 - Keep the tree balanced when possible
 - Use appropriate node structures for the specific use case

Case study – Simple File system structure

Here's an example implementing a simple file system structure:

```

class FileSystemNode(TreeNode):
    def __init__(self, name, is_directory=True):
        super().__init__(name)
        self.is_directory = is_directory

    def display(self, level=0):
        prefix = " " * level
        file_type = "dir" if self.is_directory else "file"
        print(f"{prefix} {self.data} [{file_type}]")
        for child in self.children:
            child.display(level + 1)

# Usage example
root = FileSystemNode("root")
documents = FileSystemNode("Documents")
photos = FileSystemNode("Photos")
report = FileSystemNode("report.doc", False)
vacation = FileSystemNode("vacation.jpg", False)

root.add_child(documents)
root.add_child(photos)
documents.add_child(report)
photos.add_child(vacation)

root.display()

```

General trees are flexible in representing hierarchical relationships. By understanding their implementation and traversal methods in Python, you can effectively utilize them in various applications ranging from file systems to organizational charts. As you continue your journey in programming, mastering general trees will enhance your ability to solve complex problems efficiently.

8.3 Binary Trees

In Binary tree data structure each node has left child node and right child node. This chapter will provide an in-depth exploration of binary trees, including their definition, implementation in Python, traversal techniques, practical applications, and best practices.

BINARY TREES: DEFINITION AND STRUCTURE

A node of binary tree consist of -

- A data element.
- A reference to the left child (which is also a binary tree).
- A reference to the right child (which is also a binary tree).

A sample Binary tree is depicted in the following diagram

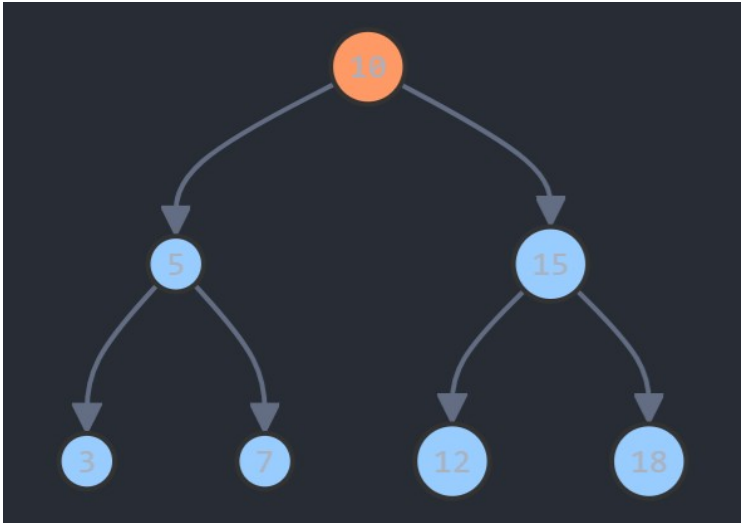


Figure 8.1 : A Binary Tree

TERMINOLOGY

- Node: The primary element that references child nodes
- Root: The main node in the tree without any parent
- Leaf: A node without children.
- Height: The longest path's length from the root node to a leaf node

TYPES OF BINARY TREES

Full Binary Tree

A full binary tree has zero or two child nodes. It is sometimes also known as a proper Binary Tree.

Complete Binary Tree

A complete binary tree has nodes at all levels except the last level.

Perfect Binary Tree

In a perfect binary tree all internal nodes have two child nodes and leaf nodes are at the same level.

TRAVERSING BINARY TREES

We can visit the nodes of the tree in in-order, pre-order, and post-order way.

IN-ORDER TRAVERSAL

The order of traversal is left node first followed by root node and then right node.

```

def in_order_traversal(node):
    if node:
        in_order_traversal(node.left)
        print(node.data)
        in_order_traversal(node.right)
  
```

Example usage

```

in_order_traversal(root)
'''
  
```

In-order traversal is depicted in the following diagram

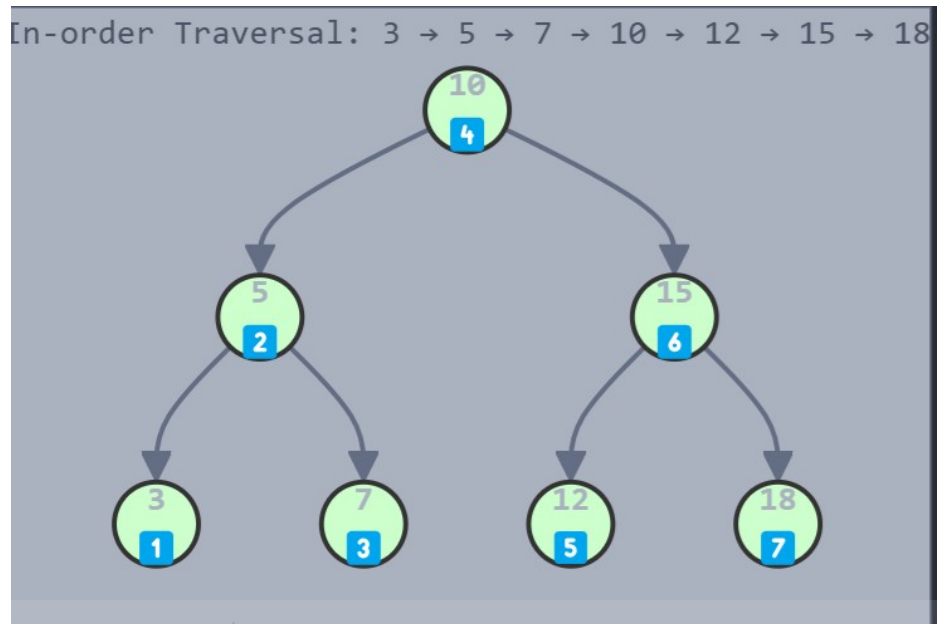


Figure 8.2 :Inorder traversal of a Binary Tree

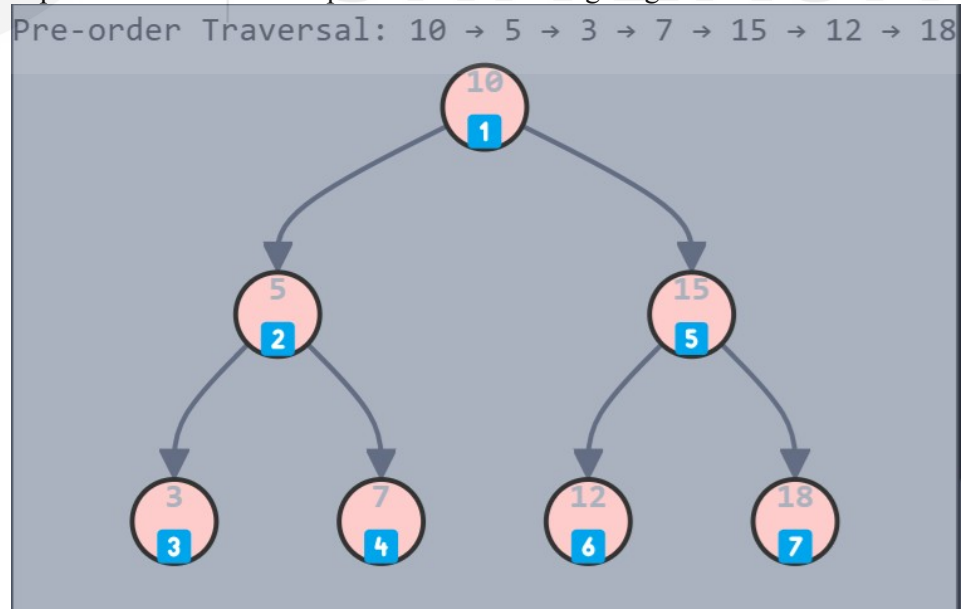
PRE-ORDER TRAVERSAL

Root node is visited first followed by its children in pre-order traversal.

```
def pre_order_traversal(node):
    if node:
        print(node.data)
        pre_order_traversal(node.left)
        pre_order_traversal(node.right)
```

```
# Example usage
pre_order_traversal(root)
'''
```

A pre-order traversal is depicted in the following diagram

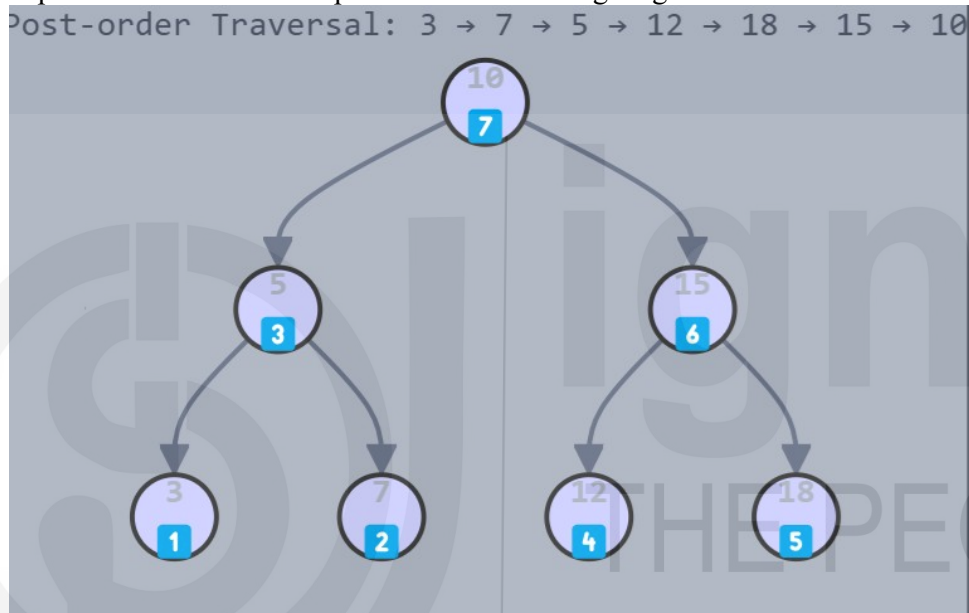


The child nodes are visited before the root node in post-order traversal.

```
def post_order_traversal(node):  
    if node:  
        post_order_traversal(node.left)  
        post_order_traversal(node.right)  
        print(node.data)
```

```
# Example usage  
post_order_traversal(root)  
...
```

A post order traversal is depicted in the following diagram



PRACTICAL USE CASES OF BINARY TREES

Binary Trees find applications in various domains:

1. File System Organization: Directory structures in file systems
2. Expression Evaluation: Mathematical expression parsing
3. Decision Trees: In machine learning for classification
4. Huffman Coding: Data compression algorithms
5. Database Indexing: B-trees and their variants

Let's implement a practical example of using Binary Trees for expression evaluation:

```
class ExpressionTree:  
    def __init__(self, data):  
        self.data = data  
        self.left = None  
        self.right = None  
  
    def evaluate(self):  
        # Leaf node (operand)  
        if not self.left and not self.right:  
            return float(self.data)
```

```

        # Evaluate left and right subtrees
        left_val = self.left.evaluate()
        right_val = self.right.evaluate()

        # Apply operator
        if self.data == '+': return left_val + right_val
        if self.data == '-': return left_val - right_val
        if self.data == '*': return left_val * right_val
        if self.data == '/': return left_val / right_val

```

BEST PRACTICES

1. When implementing Binary Trees, consider these best practices:
2. Always check for empty tree conditions
3. Use recursive solutions for tree traversal when tree depth is reasonable
4. Implement iterative solutions for very deep trees to avoid stack overflow
5. Keep track of tree balance to maintain optimal performance
6. Use appropriate tree variants based on the use case (BST for searching, AVL for balanced operations)

Time complexity for basic operations:

- Insertion: $O(n)$ for a basic binary tree
- Searching: $O(n)$ for a basic binary tree
- Traversal: $O(n)$ for all methods

Binary trees are an essential data structure that provides an efficient way to organize and manage hierarchical data. By understanding their implementation and traversal methods in Python, you can effectively utilize them across various applications ranging from search algorithms to expression parsing. Mastering binary trees will enhance your problem-solving capabilities as you progress in your programming journey.

8.4 Implementing Trees

IMPLEMENTING GENERAL TREES IN PYTHON

To implement a general tree in Python, we can create a `Node` class to represent each node in the tree. Each node will hold data and a list of its children.

```

class Node:
    def __init__(self, data):
        self.data = data
        self.children = []

    def add_child(self, child_node):
        self.children.append(child_node)

    def __repr__(self):
        return f"Node({self.data})"
...

```

To create a general tree, we can instantiate nodes and establish parent-child relationships using the `add\_child` method.

```
# Create nodes
root = Node("CEO")
cto = Node("CTO")
cfo = Node("CFO")
dev1 = Node("Developer 1")
dev2 = Node("Developer 2")

# Build the tree
root.add_child(cto)
root.add_child(cfo)
cto.add_child(dev1)
cto.add_child(dev2)
...
```

IMPLEMENTING BINARY TREES IN PYTHON

To implement a binary tree in Python, we can create a `Node` class to represent each node in the tree.

```
class Node:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None

class BinaryTree:
    def __init__(self):
        self.root = None
    ...
```

CREATING A BINARY TREE

To create a binary tree, we can instantiate nodes and establish parent-child relationships.

```
# Create nodes
root = Node(1)
root.left = Node(2)
root.right = Node(3)
root.left.left = Node(4)
root.left.right = Node(5)
...
```

BINARY TREE OPERATIONS

Let us implement the basic operations that can be performed on a Binary Tree:

```
class BinaryTree:
    def __init__(self):
        self.root = None # Initialize an empty tree
```

```

def insert(self, data):
    if not self.root:
        self.root = Node(data)
        return

    # Use queue for level order insertion
    queue = [self.root]
    while queue:
        node = queue.pop(0)

        if not node.left:
            node.left = Node(data)
            return
        else:
            queue.append(node.left)

        if not node.right:
            node.right = Node(data)
            return
        else:
            queue.append(node.right)

```

8.5 Tree Traversal Algorithms

We shall look at various traversal algorithms in this section.

TREE TRAVERSAL: DEFINITION AND IMPORTANCE

Tree traversal refers to the process of visiting all the nodes in a tree data structure in a systematic manner. The order in which nodes are visited can significantly affect the performance of operations such as searching and sorting.

TYPES OF TREE TRAVERSAL

The main types of tree traversal algorithms are:

- Depth-First Traversal: Visits nodes along a branch before backtracking.
 - Pre-order Traversal
 - In-order Traversal
 - Post-order Traversal

- Breadth-First Traversal: In this method, we visit the nodes at current depth level before visiting the nodes at next level.

IMPLEMENTING TREE TRAVERSAL ALGORITHMS IN PYTHON

To implement tree traversal algorithms, we will first define a binary tree structure using a 'Node' class.

```

class Node:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None

```

PRE-ORDER TRAVERSAL

In pre-order traversal, root node is visited first followed by left subtree and then right subtree.

```
def pre_order_traversal(node):
    if node:
        print(node.data)
        pre_order_traversal(node.left)
        pre_order_traversal(node.right)

# Example usage
root = Node(1)
root.left = Node(2)
root.right = Node(3)
pre_order_traversal(root) # Output: 1 2 3
'''
```

IN-ORDER TRAVERSAL

In in-order traversal, we first visit left subtree then root node followed by right subtree.

```
def in_order_traversal(node):
    if node:
        in_order_traversal(node.left)
        print(node.data)
        in_order_traversal(node.right)

# Example usage
in_order_traversal(root) # Output: 2 1 3
'''
```

POST-ORDER TRAVERSAL

In post-order traversal, the nodes are visited as follows: left subtree, right subtree, root.

```
def post_order_traversal(node):
    if node:
        post_order_traversal(node.left)
        post_order_traversal(node.right)
        print(node.data)

# Example usage
post_order_traversal(root) # Output: 2 3 1
'''
```

PRACTICAL USE CASES OF TREE TRAVERSAL ALGORITHMS

Tree traversal algorithms have numerous practical applications:

- Expression Evaluation: In compilers, expression trees are traversed to evaluate mathematical expressions.
- File System Navigation: Traversing directory structures to list files and folders.
- Database Indexing: Efficiently searching through hierarchical data structures used in databases.

BEST PRACTICES

When implementing tree traversal algorithms, consider these best practices:

- Use Recursion Judiciously: While recursive implementations are often simpler and more readable, be cautious of stack overflow errors with deep trees.
- Optimize for Performance: For large trees, consider iterative approaches where applicable to avoid excessive memory usage.
- Encapsulation: Keep traversal methods encapsulated within the tree class to maintain clean code architecture.

We can leverage the tree traversal algorithms to access the hierarchical data. Trees are used in various domains such as databases, file systems, and expression evaluation.

8.6 Case Study – An Expression Tree

In computer science, expression trees are a specialized form of binary trees used to represent expressions. Each internal node corresponds to an operator, while the leaf nodes represent operands. We can efficiently evaluate expressions and is widely used in compilers and interpreters. In this chapter, we will explore the concept of expression trees, their implementation in Python, and their practical applications.

EXPRESSION TREES: DEFINITION AND STRUCTURE

An expression tree is a binary tree where:

- Each internal node represents an operator (e.g., +, -, *, /).
- Each leaf node represents an operand (e.g., constants or variables).

TERMINOLOGY

- Operator: A symbol that represents an operation (e.g., addition).
- Operand: A value on which the operation is performed.
- Subtree: A tree formed by a node and its descendants.

IMPLEMENTING AN EXPRESSION TREE IN PYTHON

To implement an expression tree in Python, we will define a 'Node' class to represent each node in the tree.

```
class Node:
    def __init__(self, value):
        self.value = value
        self.left = None
        self.right = None
'''
```

CREATING AN EXPRESSION TREE

Linked List

We can create an expression tree for the expression $((a + b) * (c - d))$.
A tree would look like this:

```
'''
    *
  /\
 + -
 /\ /\
a b c d
'''
```

The implementation can be done as follows:

```
def create_expression_tree():
    root = Node('*')
    root.left = Node('+')
    root.right = Node('-')
    root.left.left = Node('a')
    root.left.right = Node('b')
    root.right.left = Node('c')
    root.right.right = Node('d')
    return root

expression_tree_root = create_expression_tree()
'''
```

EVALUATING AN EXPRESSION TREE

To evaluate the expression represented by the tree, we can use a recursive function that processes each node based on whether it is an operator or an operand.

```
def evaluate_expression_tree(node):
    if node is None:
        return 0

    # leaf node (operand)
    if node.left is None and node.right is None:
        return float(node.value) # Convert to float for arithmetic operations

    # Evaluate left and right subtrees
    left_value = evaluate_expression_tree(node.left)
    right_value = evaluate_expression_tree(node.right)

    # Perform the operation at this node
    if node.value == '+':
        return left_value + right_value
    elif node.value == '-':
        return left_value - right_value
    elif node.value == '*':
```

```

        return left_value * right_value
    elif node.value == '/':
        return left_value / right_value

# Example usage
result = evaluate_expression_tree(expression_tree_root)
print(f"The result of the expression is: {result}")

```

PRACTICAL USE CASES OF EXPRESSION TREES

Expression trees have several practical applications:

- Compilers: Used for parsing and evaluating expressions in programming languages.
- Mathematical Expression Evaluation: Facilitates efficient computation of complex mathematical expressions.
- Symbolic Computation: Enables manipulation and simplification of mathematical expressions.

BEST PRACTICES

For expression trees, consider these best practices:

- Use Clear Naming Conventions: Ensure that variable names reflect their purpose to enhance code readability.
- Error Handling: Implement error handling for cases such as division by zero or invalid operations.
- Documentation: Comment your code adequately to explain complex logic or algorithms used.

Expression trees provide a powerful way to represent and evaluate mathematical expressions efficiently.

Check Your Progress 1

1. Question: Which of the following is NOT a type of tree traversal? a) Pre-order traversal b) In-order traversal c) Sequential traversal d) Post-order traversal
 - a. Answer: c) Sequential traversal Explanation: The standard tree traversal methods are pre-order, in-order, post-order, and level-order (breadth-first). Sequential traversal is not a standard tree traversal method.
2. Question: Give the highest possible nodes at level 3 (assuming root is at level 0)? a) 4 b) 8 c) 16 d) 2
 - a. Answer: b) 8 Explanation: At any level n the highest number of nodes is 2^n . For level 3, this is $2^3 = 8$ nodes.
3. Question: A binary tree with 10 nodes has how many NULL links (empty child references)? a) 9 b) 10 c) 11 d) 12
 - a. Answer: c) 11 Explanation: Binary tree has two child nodes. For n nodes, there are $2n$ references total. The number of NULL references is $(2n - (n-1))$, where $(n-1)$ is the number of actual parent-child links. For 10 nodes: $2(10) - 9 = 11$ NULL references.

4. Question: Which type of binary tree has all internal nodes containing exactly two children? a) Complete binary tree b) Full binary tree c) Perfect binary tree d) Balanced binary tree Answer: b) Full binary tree
Explanation: A full binary tree is defined as a tree where every node has either 0 or 2 children. All internal nodes must have exactly two children.
5. Question: What is the order of visiting nodes in a pre-order traversal? a) Root, Right, Left b) Left, Root, Right c) Root, Left, Right d) Left, Right, Root
a. Answer: c) Root, Left, Right Explanation: In pre-order traversal, we visit the root node first, then traverse the left subtree, and finally traverse the right subtree.
6. Question: In an expression tree, where are the operands located? a) Only at the root b) Only at leaf nodes c) At any internal node d) Only at the second level
a. Answer: b) Only at leaf nodes Explanation: In an expression tree, operands (numbers or variables) are always located at leaf nodes, while operators (+, -, *, /) are located at internal nodes.
7. Question: What would be the level-order traversal of the following binary tree?
Copy
- ```

 1
 /\
 2 3
 /\
 4 5

```
- a) 1, 2, 3, 4, 5 b) 1, 3, 2, 5, 4 c) 4, 2, 5, 1, 3 d) 1, 2, 3, 5, 4  
a. Answer: a) 1, 2, 3, 4, 5 Explanation: Level-order traversal visits nodes level by level from left to right. Starting at root (1), then level 1 (2,3), then level 2 (4,5).
8. Question: A complete binary tree with 7 nodes has which height? a) 2 b) 3 c) 4 d) 5  
a. Answer: a) 2 Explanation: In a complete binary tree, all levels except possibly the last are completely filled. With 7 nodes, we can fill level 0 (1 node), level 1 (2 nodes), and partially fill level 2 (4 nodes), giving a height of 2.
9. Question: Which data structure is typically used to implement level-order traversal? a) Stack b) Queue c) Linked List d) Array  
a. Answer: b) Queue Explanation: Level-order traversal uses a queue data structure to keep track of nodes at each level, processing them in FIFO (First In First Out) order.
10. Question: What is the time complexity of inserting a node into a binary tree using level-order insertion? a)  $O(1)$  b)  $O(\log n)$  c)  $O(n)$  d)  $O(n^2)$   
a. Answer: c)  $O(n)$  Explanation: Level-order insertion requires traversing the tree to find the first available position, which in worst case requires visiting all nodes, giving a time complexity of  $O(n)$ .

## 8.7 Summary

Trees are versatile data structures that provide efficient ways to organize hierarchical data. The unit covered:

- General trees and binary trees with their properties and implementations
- Different types of binary trees (full, complete, perfect)

- Tree traversal algorithms (pre-order, in-order, post-order, level-order)
- Expression trees and their evaluation
- Practical applications in file systems, databases, and expression evaluation

---

## 8.8 Further Readings

---

- "Data Structures and Algorithms in Python" by Michael T. Goodrich, Roberto Tamassia, and Michael H. Goldwasser
- GeeksforGeeks: Tree Data Structure documentation
- "Python Data Structures and Algorithms" by Benjamin Baka

